Assignment 2

Optimised Matrix Multiplication and Sparse Matrices

Marcela Nieto-Kuczyńska | Big Data | ULPGC | 30 October 2026

GitHub: https://github.com/marcelakuczynska/BigData-Assignment2

Abstract

In this experiment, I compared three ways of multiplying normal dense matrices in Java and then tested whether a sparse representation could be useful when many entries are zero. I kept the inputs the same for each comparison and measured each method five times after two warm-ups. At n = 320, changing the loop order reduced median time from 123.57 ms to 19.79 ms. For matrices with 99% zeros, CSR multiplication took 0.256 ms compared with 18.00 ms for the dense method. The memory results need more care because the program records an approximate Java heap change as well as an estimated array size.

1. Introduction and questions

I used the basic matrix multiplication method in Assignment 1, with three nested loops. It works, but the way the loops read the matrices can affect how fast the program runs. I wanted to see whether simple changes to that code would actually make a difference on my computer.

I also wanted to understand when a sparse representation is useful. If most entries are zero, storing every value seems wasteful. But using indexes to keep only nonzero values also has a cost, and the result of a multiplication can contain many more nonzero entries than the inputs.

My two questions were: How much do loop reordering and cache blocking change the runtime of dense matrix multiplication? And under which tested levels of sparsity is CSR faster or smaller than a dense approach? Before running the tests, I expected loop changes to help at larger sizes and CSR to become more useful as the percentage of zeros increased.

2. Short background

For square matrices, the basic three-loop multiplication performs $O(n^3)$ arithmetic operations. Changing the loop order does not change this theoretical complexity, but it changes how the program accesses rows and columns. Cache blocking divides the computation into smaller pieces to try to reuse data while it is still close to the processor [2].

CSR (Compressed Sparse Row) stores each nonzero value, its column index and a pointer showing where each row starts. It can save storage when there are few nonzero entries, although it introduces indexing and construction overhead [3].

3. Methodology

Methods and inputs

I used Java 21 and four sequential implementations. For Part A, Baseline uses the i-j-k loop order from Assignment 1. Reordered uses i-k-j. Blocked also follows i-k-j inside tiles of 32 elements. For Part B, my CSR method multiplies matrices stored in compressed sparse row form, with its own output in CSR.

The tests used long[][] and 64-bit integer values. Dense inputs were generated with values from 0 to 9, while the sparse inputs had nonzero values from 1 to 9. A fixed java.util.Random seed was used for each size and target sparsity: 20261029 + 1009 × n + 7919 × sparsity, where sparsity is -1 for Part A. This means the compared methods received the same logical input matrices.

Setting	What I used
Dense sizes	20, 40, 80, 120, 160, 200, 320
Sparse sizes	80, 160, 240, 320
Sparse input targets	50%, 80%, 95%, 99% zero entries
Repetitions	2 warm-ups + 5 measured runs per configuration
Timer	System.nanoTime(), Java 21; output allocation included
Memory	Heap delta and estimated numeric array payload (MiB)
Laptop and environment	Windows 11; Intel Core i5-10210U; 4 physical cores, 8 logical processors; 7.72 GiB RAM (hardware recorded in Assignment 1)

What was measured

I measured the multiplication call, including the allocation of the output. Input generation, correctness checks and conversion from dense to CSR were outside the timed region. I used the median of the five measured times and the interquartile range (IQR) to show how much the times varied. The program saved every measurement to CSV.

For memory, the code requested garbage collection before each run and compared used JVM heap before and after multiplication. I also calculated the numeric array payload for three dense matrices or the arrays used by the CSR representation. These are different measurements. The heap delta is affected by JVM allocation and garbage collection, and the payload estimate excludes Java object headers and temporary structures.

Correctness and limits

Before benchmarking, the program checked a 2 × 2 multiplication with a known result and tested zero and identity matrices. Every benchmark result was then compared element by element with the basic implementation. Because the values were small integers, the results were checked exactly rather than with a floating-point tolerance.

During preparation, I used planning limits of 10 seconds per multiplication and 512 MiB of memory. The simplified program did not enforce these limits automatically, which is a limitation of the method. The largest size I attempted and completed was 320 x 320; no timeouts or allocation failures were recorded. All 53 configurations have five timed runs and five heap observations. I did not measure peak process RSS or CSR conversion time.

4. Results — Part A: dense optimisation

The graph shows the median execution time for the three dense methods. The logarithmic vertical axis makes the smaller values easier to see. All three methods use the same input matrices for each size.

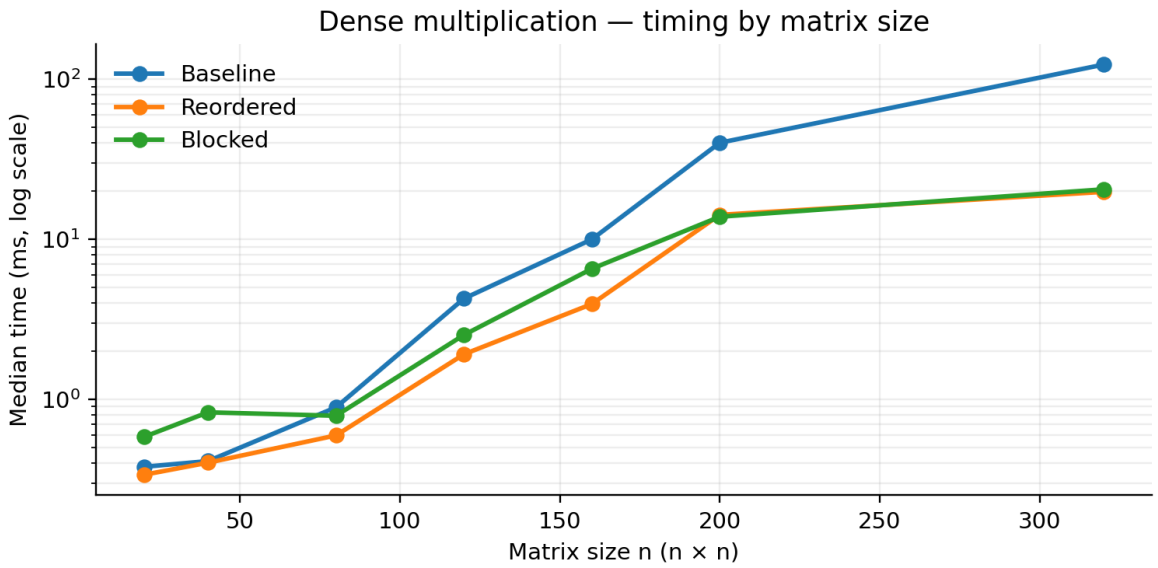


Figure 1. Dense multiplication runtime. Each point is the median of five timed runs; variability is shown in the accompanying table.

n	Baseline (ms)	Reordered (ms)	Blocked (ms)
20	0.38	0.34	0.58
40	0.41	0.40	0.83
80	0.89	0.60	0.79
120	4.26	1.91	2.53
160	10.00	3.94	6.55
200	40.13	14.23	13.84
320	123.57	19.79	20.54

At n = 320, Baseline took 123.57 ms, Reordered took 19.79 ms and Blocked took 20.54 ms. The reordered version was about 6.2 times faster than the baseline. This is the clearest result from Part A.

I expected blocking to perform well, but it was not always the fastest method. At n = 200 it was slightly faster than Reordered (13.84 vs 14.23 ms), while at n = 320 Reordered was slightly ahead. Smaller sizes had much less stable timings. The IQR at n = 320 was 18.56 ms for Baseline, 1.35 ms for Reordered and 4.30 ms for Blocked.

The two optimisations still carry out $O(n^3)$ arithmetic work. What changed is how the code accessed the data. My results support the idea that this can matter a lot, but they do not establish that one loop order or block size will always win.

5. Results — Part B: sparse matrices

For the second part, I compared CSR with the Reordered dense method using identical logical inputs. "Sparsity" means the target percentage of zeros in each input matrix. The actual nonzero counts were recorded separately, because random generation does not produce an exact percentage every time.

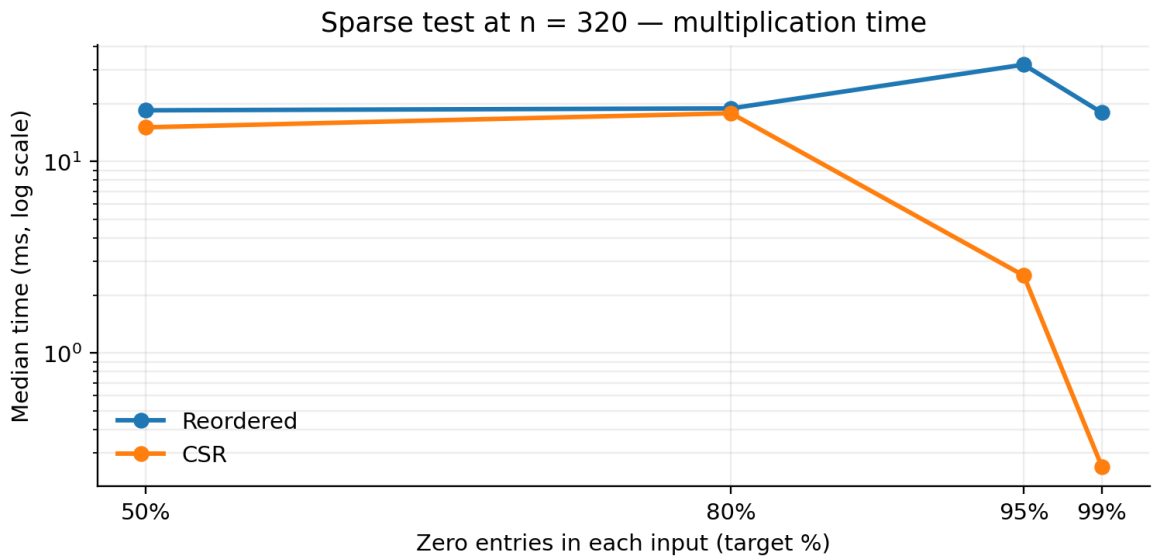


Figure 2. Median multiplication time at n = 320. CSR input conversion is excluded from the measured interval.

Target zeros	Dense (ms)	CSR (ms)	CSR speedup	Nonzeros in C
50%	18.504	15.080	1.2×	102,400
80%	18.936	17.861	1.1×	102,399
95%	32.048	2.538	12.6×	55,364
99%	17.998	0.256	70.4×	2,931

The difference is easy to see at high sparsity. At 95% zeros, CSR was about 12.6 times faster than Reordered; at 99% zeros, it was about 70.4 times faster. At 50% and 80% zeros, the advantage at n = 320 was small, and not every smaller test favoured CSR.

This is why I would not choose CSR based only on knowing that a matrix contains zeros. At 80% zeros, the result matrix at n = 320 still had 102,399 nonzero entries out of 102,400. Even though both inputs were sparse, their product was effectively dense, so the CSR method still had to store almost everything.

6. Looking at all sparse configurations

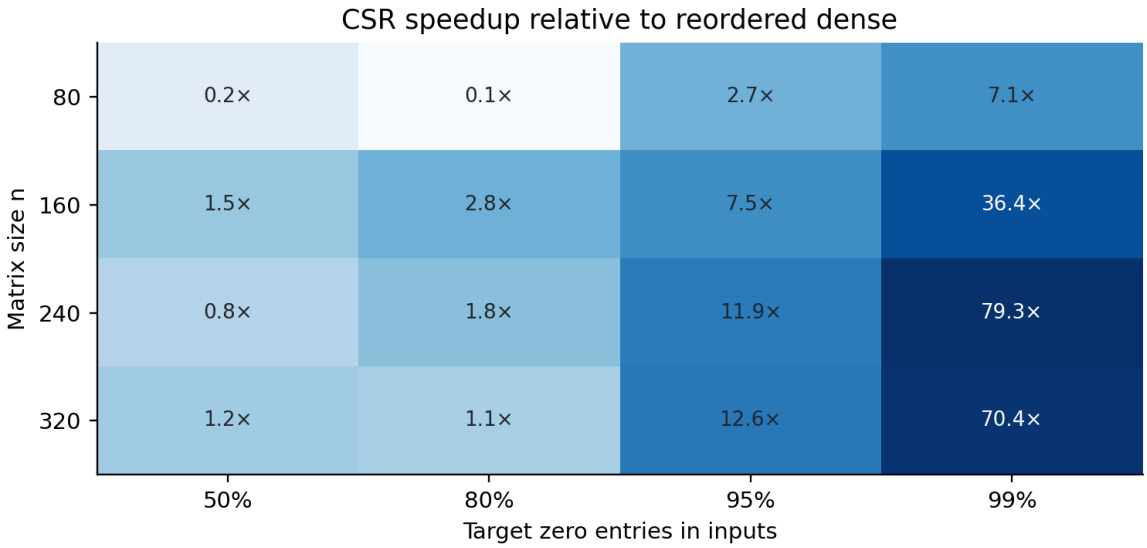


Figure 3. CSR speedup = dense Reordered median / CSR median. Values below 1 mean CSR was slower.

At n = 80, CSR was slower than Reordered with 50% or 80% zeros. By 99% zeros, it was faster for every tested size. I think this is a useful result because it shows that the point where sparse multiplication helps depends on both the matrix size and the input structure.

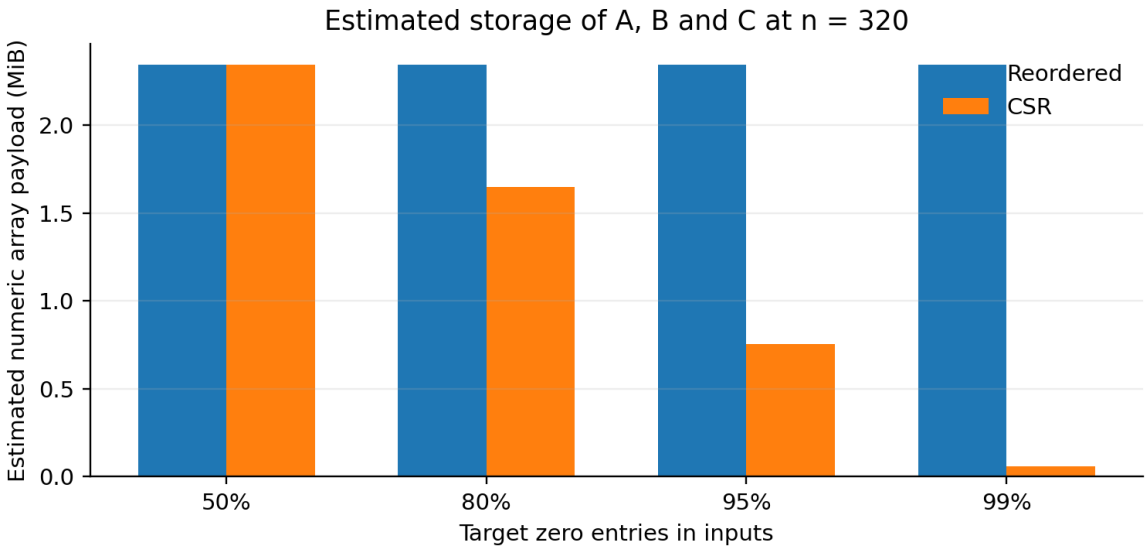


Figure 4. Estimated payload of the numeric arrays for A, B and C at n = 320. These are not measured JVM heap values.

For n = 320 and 99% zeros, the estimated numeric payload was 0.060 MiB for CSR, compared with 2.344 MiB for the dense representation. But the retained heap deltas were 0.982 MiB and 0.827 MiB respectively. The difference reminds me that the CSR algorithm also creates temporary arrays and list objects while building its result. The reported payload estimate does not include those temporary objects.

7. Discussion and limitations

The dense results show that I could get a large speed improvement without changing the mathematical operation or introducing threads. Simply going through the matrices in a different order made a visible difference at larger sizes. Cache blocking also helped compared with the baseline, but with these inputs and a block size of 32, it was not consistently faster than simple reordering.

The sparse results were more mixed. CSR performed very well when there were very few nonzero entries. With less sparsity, the indexing and result-building work reduced its advantage. Also, a sparse input does not guarantee a sparse output, which the nonzero counts in my experiment make clear.

There are limits to these results. My tested matrices were at most 320 × 320, generated randomly, with small integer values and a single JVM process. The Java runtime can optimise the code during execution, and some measured times varied substantially. I measured the kernel rather than the full workflow, so the CSR advantage does not include converting data to CSR in the first place. I also did not test different block sizes, real-world sparsity patterns or other machines.

The heap-delta results are especially approximate. Garbage collection and short-lived allocations make them less reliable than an actual process-memory profile. The storage chart is useful for comparing the numeric arrays, but it does not represent total JVM memory. I used the same laptop as in Assignment 1, so I reused the hardware details recorded there. The CPU model, core counts and RAM were not measured again during Assignment 2.

8. Conclusion

For this experiment, the best dense result at n = 320 came from changing the loop order. CSR became clearly useful at the highest sparsity levels, while the results were much less convincing when there were fewer zeros.

References and materials

[1] Universidad de Las Palmas de Gran Canaria, Individual Assignments: Matrix Multiplication, Big Data 2026–2027, Assignment 2 and common protocol, pp. 2–5, 8.

[2] Cornell University, CS 6210: Matrix Computations, lecture on matrix multiplication and blocking (2025). https://www.cs.cornell.edu/courses/cs6210/2025fa/lec/2025-09-03.html

[3] SciPy, csr_matrix documentation. https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csr_matrix.html

[4] Oracle, System.nanoTime(), Java SE 21 API. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/System.html

Materials and reproducibility

The Java code, input settings, raw measurements, summary CSV and plotting script are included in the Assignment 2 materials. I reused the basic Java multiplication method from Assignment 1, which also contains the details of the laptop used for the experiments: https://github.com/marcelakuczynska/BigData-Assignment1

AI use

I used ChatGPT to help with the Java implementation, debugging and plotting scripts. I ran the experiments on my own laptop and reviewed the results myself.